

Backend Challenge — Transaction Execution API

¡Hola! Bienvenido(a) al challenge técnico de backend de **Spin**. Estamos emocionados de que estés en este proceso y queremos conocer tu forma de pensar y construir software.

El reto

Vas a construir un **API REST** que gestione la ejecución de transacciones financieras (crédito y débito). Tu servicio será responsable de:

1. **Recibir solicitudes** de transacciones (crédito o débito).
2. **Ejecutar** la transacción contra un proveedor externo que controla los bloqueos y balance de la cuenta.
3. **Persistir** el resultado.
4. **Exponer** un endpoint de consulta para recuperar transacciones.

Contexto: El sistema opera en un entorno de alto volumen — se esperan millones de transacciones diarias. Tus decisiones de diseño deben considerar escalabilidad y rendimiento.

Nota clave: No necesitas implementar lógica de gestión de balances y cuentas. El proveedor externo es quien valida y gestiona los saldos y bloqueos. Tu servicio solo ejecuta, registra y expone transacciones.

Endpoints requeridos

1. Ejecutar Transacción

Método	POST
Path	/transactions

Request body:

```
{  
  "accountId": "acc-123456",  
  "type": "CREDIT",  
  "amount": 1500.00,  
}
```

```
"currency": "MXN",  
"description": "Transferencia recibida"  
}
```

Tipos soportados: CREDIT | DEBIT

Flujo esperado:

1. Validar el request de entrada.
2. Aplicar las **reglas de negocio** (ver sección "Reglas de Negocio").
3. Llamar al **proveedor externo** para ejecutar la transacción (ver sección "Proveedor Externo").
4. Persistir la transacción con el resultado del proveedor (incluyendo el saldo actualizado que retorna el proveedor).
5. Retornar la transacción creada con su estado.

Response esperado (ejemplo):

```
{  
  "id": "uuid-generado",  
  "accountId": "acc-123456",  
  "type": "CREDIT",  
  "amount": 1500.00,  
  "currency": "MXN",  
  "description": "Transferencia recibida",  
  "status": "EXECUTED",  
  "providerTransactionId": "txn-789",  
  "balanceAfter": 5500.00,  
  "createdAt": "2025-03-15T10:30:00Z"  
}
```

2. Consultar Transacciones

Método	GET
Path	/transactions
Query params opcionales	accountId, status, type, page, limit

Retorna la lista de transacciones almacenadas. Se valora implementar filtros y paginación.

 **Proveedor externo (mock)**

Tu servicio debe comunicarse con un proveedor externo para ejecutar cada transacción. **No necesitas implementar este proveedor**; debes mockearlo de la forma que consideres más adecuada (WireMock, Postman, mock server, stub configurable, etc.).

Contrato del proveedor

POST /provider/v1/execute

Request:

```
{
  "accountId": "acc-123456",
  "type": "CREDIT",
  "amount": 1500.00,
  "currency": "MXN"
}
```

Response exitosa (HTTP 200):

```
{
  "transactionId": "txn-789",
  "status": "APPROVED",
  "balance": 5500.00,
  "executedAt": "2025-03-15T10:30:00Z"
}
```

Response fallida (HTTP 4XX - 5XX):

```
{
  "status": "REJECTED",
  "code": "INSUFFICIENT_FUNDS",
  "message": "The account does not have enough balance to complete the transaction"
}
```

Tu servicio debe manejar ambos escenarios correctamente y reflejar el resultado en la transacción persistida.

Reglas de Negocio

Tu servicio debe implementar las siguientes validaciones **antes** de ejecutar la transacción contra el proveedor externo:

1. **Monto mínimo:** Toda transacción debe tener un monto mayor a **\$1.00**. Rechazar con error si el monto es menor o igual.
2. **Monto máximo por transacción:** Las transacciones de tipo **DEBIT** no pueden exceder **\$10,000.00** por operación. Las de tipo **CREDIT** no tienen límite.

3. **Moneda soportada:** Solo se aceptan transacciones en **MXN**. Cualquier otra moneda debe ser rechazada.

Importante: Estas validaciones deben ocurrir en tu servicio antes de llamar al proveedor externo. Si la transacción no cumple las reglas, no se debe ejecutar y se debe retornar un error descriptivo al cliente.

Requerimientos técnicos

Aspecto	Detalle
Lenguaje	El que te sientas más cómodo(a). Queremos ver tu mejor código, no luchar con un lenguaje que no dominas.
Persistencia	A tu elección (in-memory, SQLite, PostgreSQL, MongoDB, etc.). Justifica tu decisión en el README.
Testing	Se espera al menos tests unitarios.
Documentación	README.md con instrucciones claras para levantar y ejecutar el proyecto.

Entregables

1. **Código fuente** en un repositorio Git.
2. **README.md** con instrucciones para ejecutar el proyecto y decisiones de diseño.
3. **Nota sobre uso de IA** (ver sección abajo).

Todo lo adicional que decidas incluir (tests, Dockerfile, documentación extra, OpenAPI spec, etc.) es bienvenido y será considerado.

¿Qué valoramos?

- **Código limpio y legible** — Naming claro, funciones con responsabilidad única, código que comunica intención.
- **Arquitectura bien organizada** — Separación de capas, inversión de dependencias, estructura coherente del proyecto.
- **Testing** — Cobertura razonable del dominio y mocks del proveedor externo.
- **Manejo del proveedor externo** — Desacoplamiento del cliente HTTP; resiliencia (timeouts, reintentos, circuit-breaker) es un plus.
- **Pragmatismo** — No sobre-ingenierizar; resolver de forma simple y extensible.
- **Documentación** — README útil, decisiones de diseño explicadas.

Aplica las mejores prácticas que conozcas. Esperamos un código con la calidad que usarías en un entorno profesional de producción.

Uso de Inteligencia Artificial

El uso de herramientas de IA (Copilot, ChatGPT, Claude, etc.) está permitido. La IA es parte del flujo de trabajo moderno y no penalizamos su uso.

Te pedimos que:

1. **Documentes en el README** si usaste IA y de qué forma (generar boilerplate, resolver dudas, debugging, etc.).
2. **Entiendas y puedas explicar** cada línea del código que entregues — en la siguiente etapa del proceso podremos hacerte preguntas sobre tus decisiones.

La IA es una herramienta; el criterio de ingeniería sigue siendo tuyo.

Plazo de entrega

3 días calendario desde la recepción de este challenge.

Envía el link a tu repositorio (si es privado, otorga acceso al correo que te indicará el equipo de People).

¿Por qué Spin?

- Trabajamos con **arquitectura de microservicios** en un entorno fintech real de alto impacto.
- Nuestro stack incluye **Go, Java, Kafka, MongoDB, PostgreSQL, Kubernetes**.
- Valoramos ingenieros que escriben **código con intención** — que no sólo funcione, sino que comunique.
- La IA es aliada, no enemiga — la usamos para **amplificar** el talento del equipo.

Que la Fuerza te acompañe en este challenge. 🙌